

ACT 2 Parte Práctica Mini Menú Pausa

2n DAM-Vi Oliver Martínez Carbonell

Repositorio:

https://github.com/a25olimarcar-afk/UF1_ACT2_MiniMenu_OMC/releases/tag/v1.0

Menú Pausa

Escena de simulación + Jerarquía de clase

La escena de simulación consta de una pelota que rebota de forma indefinida sobre diferentes plataformas que la rodean para que esta no pueda escapar.

La jerarquía se a construido al igual que se ha especificado en clase, un objeto Canvas, y debajo de este colgara nuestro otro objeto que representara el menú de Pausa, el cual tendrá un componente Canvas Group. Dentro de este objeto PauseCanvas se encuentra toda la interfaz.



Pausa con Time.timeScale + PauseMenu

Aquí podemos observar el método con el que abrimos y cerramos el menú de pausa, el método OnPauseMenu() se comprueba todo el rato desde Update, al recibir la entrada Escape, se comprueba con un booleano si se a de abrir o cerrar el menú, podemos ver también como se aplica Time.timeScale = 0.0f cuando el menú está abierto, deteniendo así la simulación de nuestra escena.

```
1 referencia
private void OnPauseMenu()
{
    if (Input.GetKeyDown(KeyCode.Escape))
    {
        if (isOpen == false) //si el menu de pausa no esta abierto, lo abrimos
        {
            canvasFader.FadeIn(0.5f);
            Time.timeScale = 0.0f;

            //al poder ver el menu, lo hacemos interactuable
            canvasFader.GetCanvasGroup().interactable = true;

            isOpen = true; //luego marcamos que esta abierto
        }
        else //si el menu de pausa esta abierto, lo cerramos
        {
            //pero, antes de cerrarlo, comprobamos si hay cambios sin guardar
            if (HasUnsavedChanges())
            {
                ShowWarning();
            }
            else
            {
                canvasFader.FadeOut(0.5f);
                Time.timeScale = 1.0f;
                //al cerrar el menu, lo hacemos no interactuable
                canvasFader.GetCanvasGroup().interactable = false;
                isOpen = false; //ahora marcamos que esta cerrado
            }
        }
    }
}
```

Fade con corrutinas + Canvas Group

Arriba en el método OnPauseMenu() ya se puede observar como se llama a canvasFader para realizar transiciones tanto para cerrar como para abrir el menú de pausa. Toda la lógica viene de este script canvasFader, donde cogemos el componente CanvasGroup de Pause Canvas.

Utilizamos una corrutina y un lerp para conseguir el fade, y nos aseguramos de utilizar Time.unscaledDeltaTime en vez de Time.deltaTime, ya que este también se detiene si Time.timeScale es 0. La razón por la que maxAlpha es 0.8 y no 1.0, es porque quería que aunque el menú estuviese completamente abierto, pudiésemos ver como la simulación permanecía detenida.

```
using System.Collections;
using UnityEngine;

[RequireComponent(typeof(CanvasGroup))]
public class CanvasFader : MonoBehaviour
{
    [SerializeField] private CanvasGroup canvasGroup;

    [SerializeField] private float maxAlpha = 0.8f;
    [SerializeField] private float minAlpha = 0.0f;

    private void Awake()
    {
        canvasGroup = GetComponent<CanvasGroup>();
        canvasGroup.alpha = 0f;
    }

    public CanvasGroup GetCanvasGroup()
    {
        return canvasGroup;
    }

    public void FadeIn(float duration) //metodo para fade in
    {
        StartCoroutine(FadeRoutine(canvasGroup.alpha, maxAlpha, duration));
    }

    public void FadeOut(float duration) //metodo para fade out
    {
        StartCoroutine(FadeRoutine(canvasGroup.alpha, minAlpha, duration));
    }

    private IEnumerator FadeRoutine(float startAlpha, float endAlpha, float duration) //corutina para realizar el fade
    {
        float elapsedTime = 0f; //tiempo transcurrido desde que se inicio la corutina

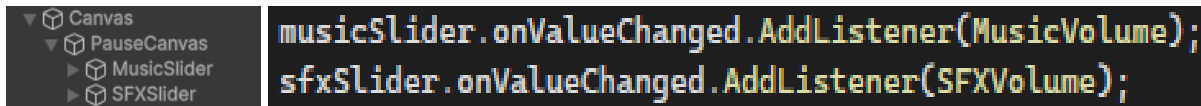
        while (elapsedTime < duration) //mientras el tiempo transcurrido sea menor a la duración del fade
        {
            //utilizar unscaledDeltaTime significa que el tiempo utilizado por la animación ignora Time.timeScale, así evitamos problemas
            //esto es porque cuando Time.timeScale es 0, Time.deltaTime también pasa a ser 0
            elapsedTime += Time.unscaledDeltaTime;
            //hacemos un Lerp entre el alpha inicial y el final, en función del tiempo transcurrido y la duración
            canvasGroup.alpha = Mathf.Lerp(startAlpha, endAlpha, elapsedTime / duration);
            yield return null; //esperamos un frame
        }

        canvasGroup.alpha = endAlpha; //al terminar la corutina, aseguramos que el alpha final sea exacto
    }
}
```

Sliders

Slider Música + Slider SFX

Dentro de nuestro PauseCanvas tenemos dos Sliders. Después de agregar sus respectivos Listeners en Start, llamarán a estos métodos que se encargan respectivamente de actualizar y guardar la musica o los SFX.



```
1 referencia
private void MusicVolume(float vol)
{
    musicAudioSource.volume = vol;
    musicVolumeText.text = vol.ToString("0.00"); //actualizamos el texto del volumen de la musica

    //guardamos el valor temporalmente, pero todavía no en PlayerPrefs
    tempMusicVolume = vol;

    Debug.Log($"Music volume set to: {vol}");
}

1 referencia
private void SFXVolume(float vol)
{
    sfxAudioSource.volume = vol;
    ballAudio.volume = vol;
    sfxVolumeText.text = vol.ToString("0.00");

    tempSFXVolume = vol;

    Debug.Log($"SFX volume set to: {vol}");
}
```

Guardar en PlayerPrefs + Actualizar Sliders

Pero como está implementado el extra del boton Apply, tenemos este método ApplySettings() que es el que realmente guarda estos valores y todos los demás. En los otros métodos solo guardamos un valor temporal. Después también tenemos el método ObtainAndUpdateSliders() que se llama en Start y se encarga actualizar los sliders al iniciar el juego dependiendo de lo que se allá guardado.

```
1 referencia
private void ApplySettings()
{
    //al aplicar los cambios, guardamos los valores temporales en PlayerPrefs
    PlayerPrefs.SetFloat("MusicVolume", tempMusicVolume);
    PlayerPrefs.SetFloat("SFXVolume", tempSFXVolume);
}

1 referencia
private void ObtainAndUpdateSliders()
{
    //Music volume
    //obtenemos el valor guardado (si no hay ninguno usamos 1.0 predeterminado)
    float savedMusicVolume = PlayerPrefs.GetFloat("MusicVolume", 1.0f);
    tempMusicVolume = savedMusicVolume;

    //actualizar el volumen de la musica, slider y texto con el valor guardado
    musicSlider.value = savedMusicVolume;
    musicAudioSource.volume = savedMusicVolume;
    musicVolumeText.text = savedMusicVolume.ToString("0.00");

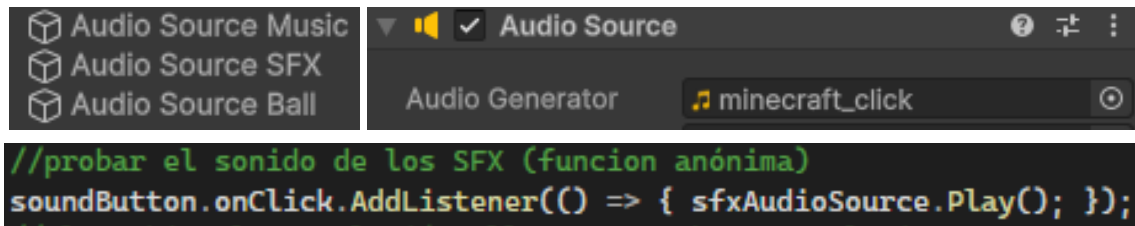
    //SFX volume
    float savedSFXVolume = PlayerPrefs.GetFloat("SFXVolume", 1.0f);
    tempSFXVolume = savedSFXVolume;

    sfxSlider.value = savedSFXVolume;
    sfxAudioSource.volume = savedSFXVolume;
    ballAudio.volume = savedSFXVolume;
    sfxVolumeText.text = savedSFXVolume.ToString("0.00");
}
```

Sonidos

Música de fondo + Botón de sonido

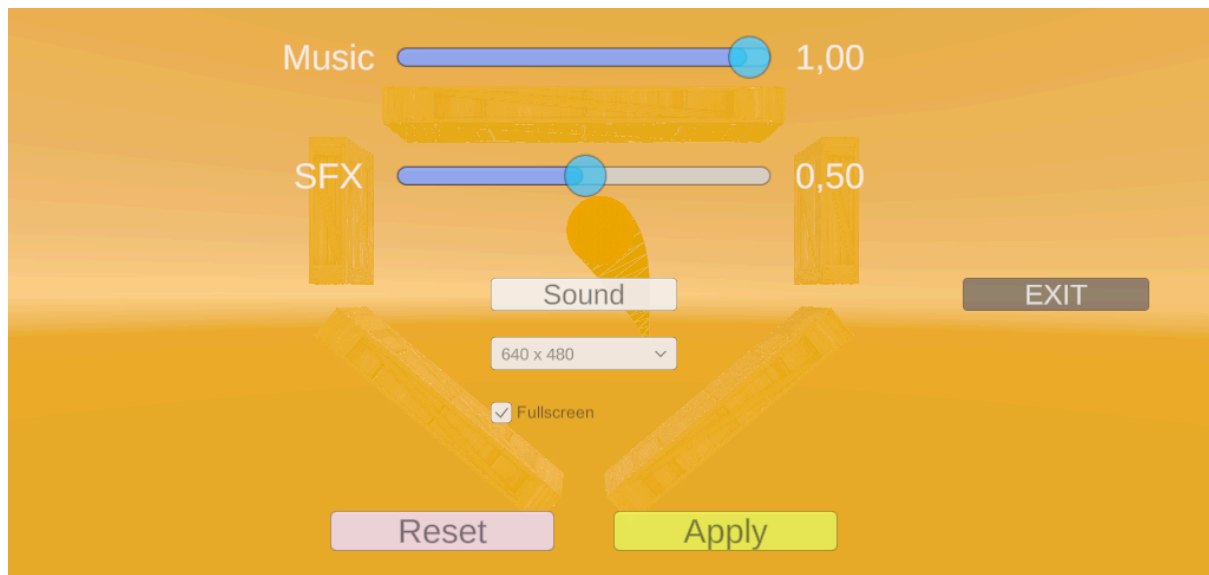
Dentro de la escena tenemos diferentes Audio Sources, uno representa la música que se escuchara de fondo y el otro el sonido que podremos reproducir con el botón. Como la lógica para lograr esto es que simplemente al hacer clic al botón se reproduzca un sonido, en vez de hacer un método podemos hacer una función anónima que lo hará todo directamente en una sola línea. Hay un sonido extra implementado que actúa como si fuese otro SFX, solo que este se utiliza cada vez que la pelota de la simulación colisiona con algo.



Modificar volumen con Sliders + Aplicar y guardar en PlayerPrefs

La modificación del volumen con los Sliders ya se ha mostrado arriba en una foto donde se podían ver los métodos MusicVolume y SFXVolume, que se encargaban de que al modificar el valor del Slider, este se actualizara y se guardase el resultado en una variable Temporal. Lo mismo sucede al guardar en PlayerPrefs y actualizar, todo se guarda definitivamente en ApplySettings() y luego ObtainAndUpdateSliders() se utiliza en Start para actualizar todos los Sliders al iniciar el juego.

Aquí se ve el menú de pausa, donde podemos ver el botón en el que pone "Sound", al hacer clic sobre ese botón se reproducirá el Audio Source SFX, y el volumen bajará dependiendo del valor aplicado en el Slider SFX.



Esta imagen también muestra toda la interfaz que contiene todos los puntos de la rúbrica y los extra. Además de un botón para salir del juego por si acaso.

Dropdown

Obtener resoluciones disponibles del dispositivo + Filtrar tasas de refresco

Obtenemos las diferentes resoluciones a través de un método largo llamado ObtainResolutions(), este se llama en Start para poderlos obtener desde el principio del juego. Lo que hacemos en este método, es crear un array de todas las resoluciones disponibles, luego las recorremos una por una y mediante un HashSet que actuara como un filtro, ya que solo puede guardar elementos no duplicados, conseguiremos que se registren correctamente las resoluciones disponibles sin duplicados por las tasas de refresco.

```
1 referencia
private void ObtainResolutions()
{
    //limpiar las opciones predeterminadas del dropdown
    resolutionDropdown.ClearOptions();

    //obtenemos todas las resoluciones disponibles
    Resolution[] resolutions = Screen.resolutions;

    int savedWidth = PlayerPrefs.GetInt("ResolutionWidth", Screen.currentResolution.width);
    int savedHeight = PlayerPrefs.GetInt("ResolutionHeight", Screen.currentResolution.height);

    tempResolutionWidth = savedWidth;
    tempResolutionHeight = savedHeight;

    //para evitar que se repitan resoluciones, usamos un HashSet
    //guardamos las resoluciones que ya hemos añadido al dropdown
    HashSet<string> addedResolutions = new HashSet<string>();

    for (int i = 0; i < resolutions.Length; i++)
    {
        Resolution res = resolutions[i];
        string resolutionName = res.width + " x " + res.height;

        //si la resolución todavía no ha sido añadida
        if (!addedResolutions.Contains(resolutionName))
        {
            //la añadimos al conjunto de resoluciones utilizadas
            addedResolutions.Add(resolutionName);
            //la añadimos a la lista de resoluciones disponibles
            availableResolutions.Add(res);
            //y la añadimos al dropdown
            resolutionDropdown.options.Add(new TMP_Dropdown.OptionData(resolutionName));
        }
    }

    //buscamos la resolución guardada y la seleccionamos en el dropdown
    for (int i = 0; i < resolutionDropdown.options.Count; i++)
    {
        if (resolutionDropdown.options[i].text == savedWidth + " x " + savedHeight)
        {
            resolutionDropdown.value = i; //aplicar la resolución guardada en el dropdown
            break; //salimos del bucle una vez que encontramos la resolución guardada
        }
    }

    //actualizamos el valor mostrado en el dropdown
    resolutionDropdown.RefreshShownValue();
}
```

Aplicar resoluciones al Dropdown

Para que el Dropdown no sea simplemente visual y realmente permita cambiar la resolución, utilizamos el método `ChangeResolution` el cual se llamara siempre que el Dropdown detecte cambios en su valor. Este método coge el índice que hemos seleccionado dentro de nuestro Dropdown, el cual ya está filtrado, de esta forma podemos añadir la resolución de la pantalla y aplicarla.

```
1 referencia
private void ChangeResolution(int index)
{
    //obtenemos la resolución porque Unity nos devuelve un index
    //Resolution resolution = Screen.resolutions[index];

    //obtenemos la resolución de nuestra lista filtrada
    Resolution resolution = availableResolutions[index];

    Screen.SetResolution(
        resolution.width,
        resolution.height,
        Screen.fullScreen
    );

    //guardamos la resolución solamente de forma temporal
    tempResolutionWidth = resolution.width;
    tempResolutionHeight = resolution.height;
}
```

Guardar en PlayerPrefs + Actualizar pantalla y Dropdown

Pero esta solo se guardará de forma temporal, luego podremos aplicarla con `ApplySettings()`.

```
//PlayerPrefs.SetInt("ResolutionIndex", index);
//en vez de guardar el index, es mejor guardar la resolución en sí
//por si cambian las resoluciones disponibles entre dispositivos
PlayerPrefs.SetInt("ResolutionWidth", tempResolutionWidth);
PlayerPrefs.SetInt("ResolutionHeight", tempResolutionHeight);
```

Toggle

Cambiar de fullscreen a ventana

Para esto tenemos, `ChangeScreenMode`, que se encarga de comprobar si el toggle está activo o desactivado para aplicar el modo de fullscreen o ventana. Y luego esta `ObtainScreenMode` que se llama en `Start` y nos sirve para obtener el estado guardado de `PlayerPrefs`.

```
1 referencia
private void ChangeScreenMode(bool isFullscreen)
{
    if (isFullscreen) //si el toggle esta activado, ponemos pantalla completa
    {
        Screen.fullScreenMode = FullScreenMode.FullScreenWindow;
    }
    else //si el toggle esta desactivado, ponemos ventana
    {
        Screen.fullScreenMode = FullScreenMode.Windowed;
    }
    //guardamos el modo de forma temporal
    tempFullscreen = isFullscreen;
}

1 referencia
private void ObtainScreenMode()
{
    //obtenemos el estado del toggle guardado en PlayerPrefs
    int savedFullscreen = PlayerPrefs.GetInt("IsFullscreen", 1);
    bool isFullscreen = savedFullscreen == 1; //convertimos el int a bool

    tempFullscreen = isFullscreen;

    toggleScreen.isOn = isFullscreen;

    if (isFullscreen)
    {
        Screen.fullScreenMode = FullScreenMode.FullScreenWindow;
    }
    else
    {
        Screen.fullScreenMode = FullScreenMode.Windowed;
    }
}
```

Guardar en PlayerPrefs

Como este valor booleano `tempFullscreen` es solo temporal, realmente se aplicara a `PlayerPrefs` una vez se llame al método `ApplySettings()`. Como `PlayerPrefs` no puede aceptar booleanos le pasaremos el valor como si fuese un int utilizando un ternario, donde 1 será true y 0 false.

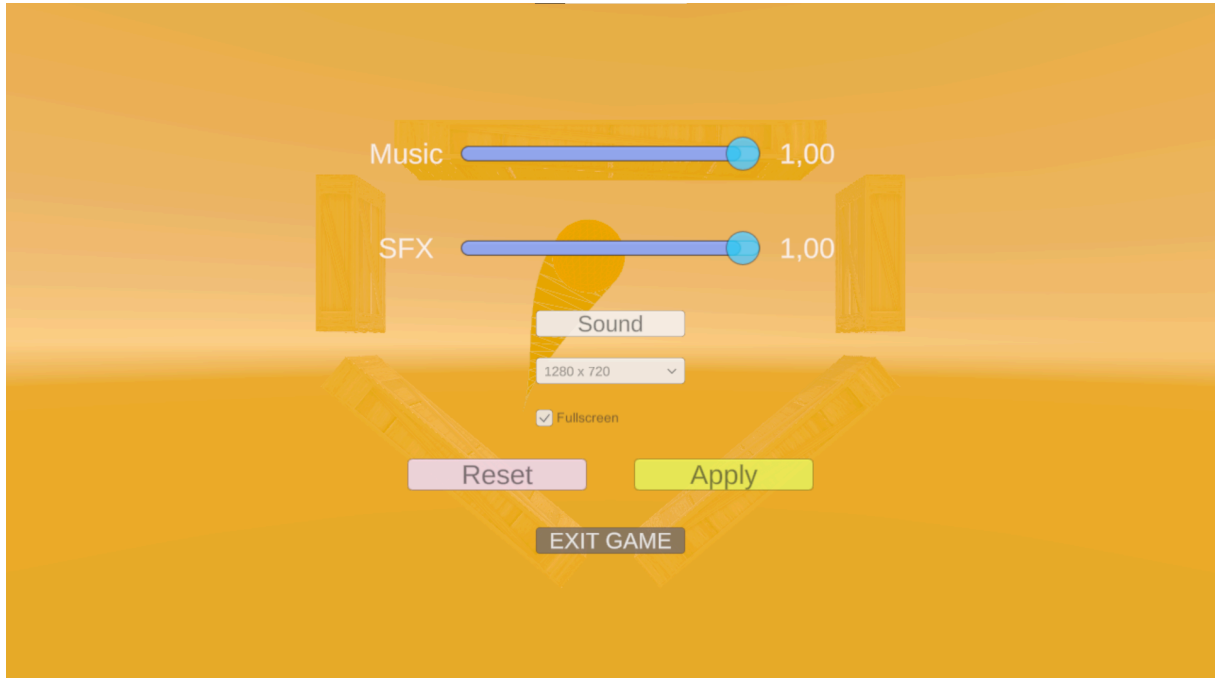
```
//guardamos el estado del toggle en PlayerPrefs
//tenemos que utilizar un int porque PlayerPrefs no soporta bools
PlayerPrefs.SetInt("IsFullscreen", tempFullscreen ? 1 : 0); //1 = true, 0 = false
```

Responsive UI

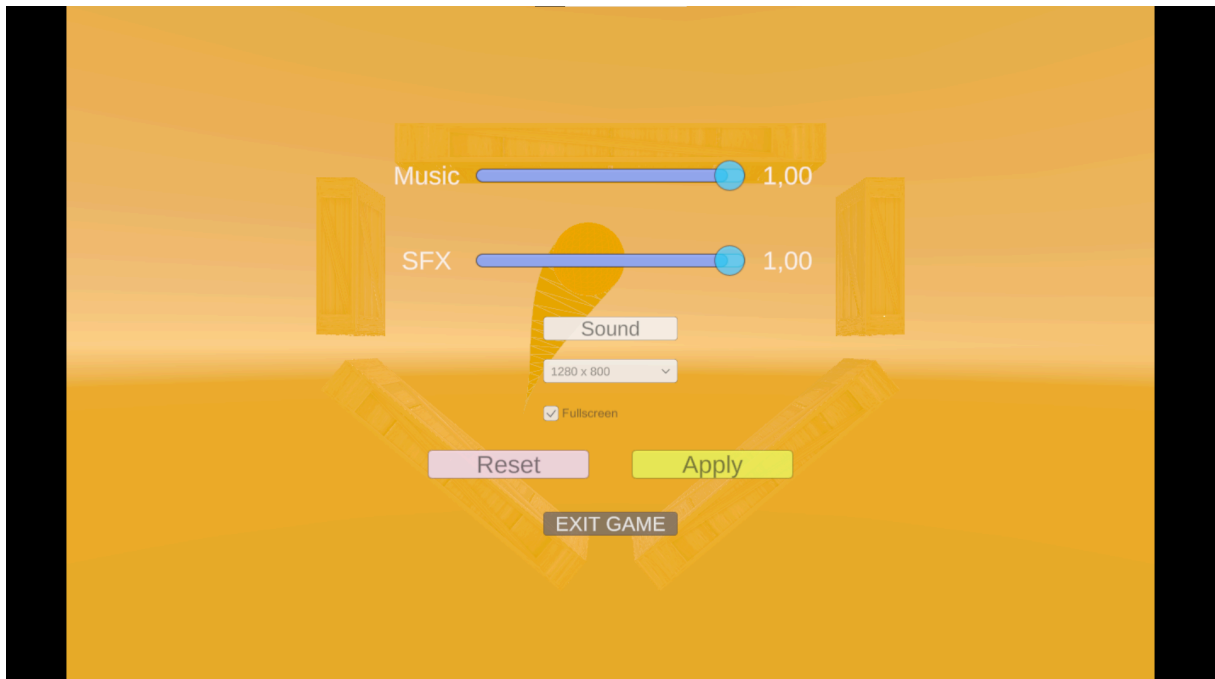
La interfaz funciona con aspect ratios 16:9 y 16:10

Aquí se muestran imágenes de la build del juego, donde se está jugando con las siguientes resoluciones de aspect ratio 16:9 y 16:10.

resolución 1280 x 720 (una de aspect ratio 16:9)



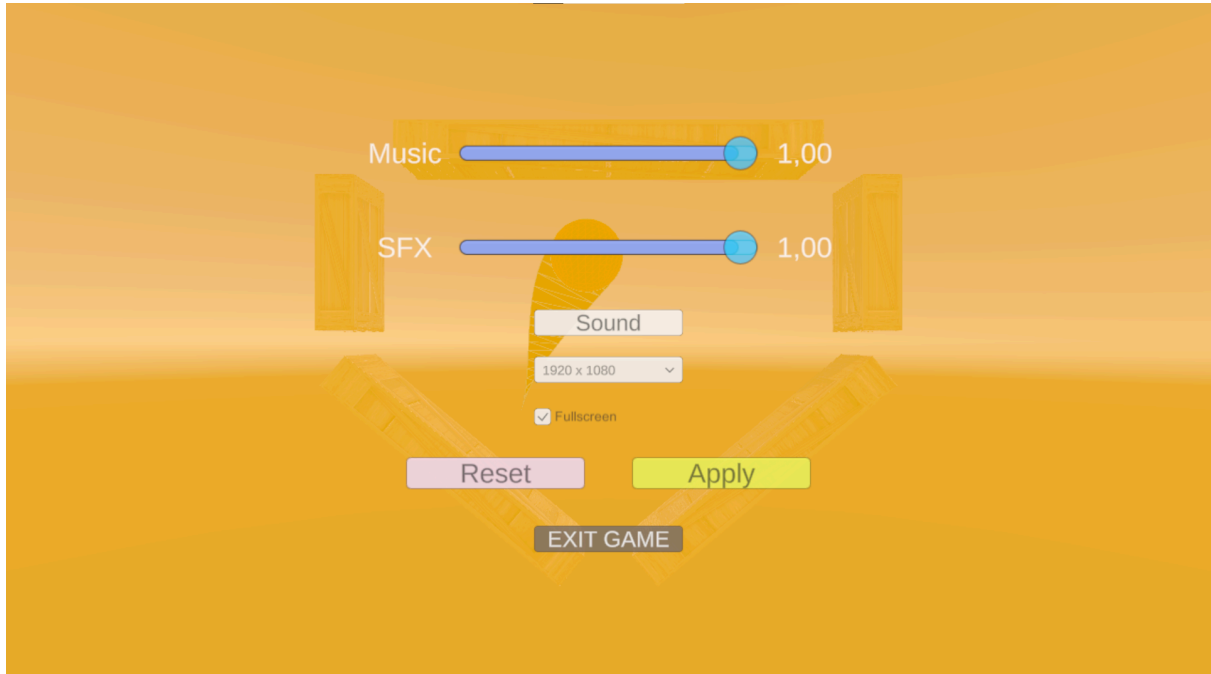
resolución 1280 x 800 (una de aspect ratio 16:10)



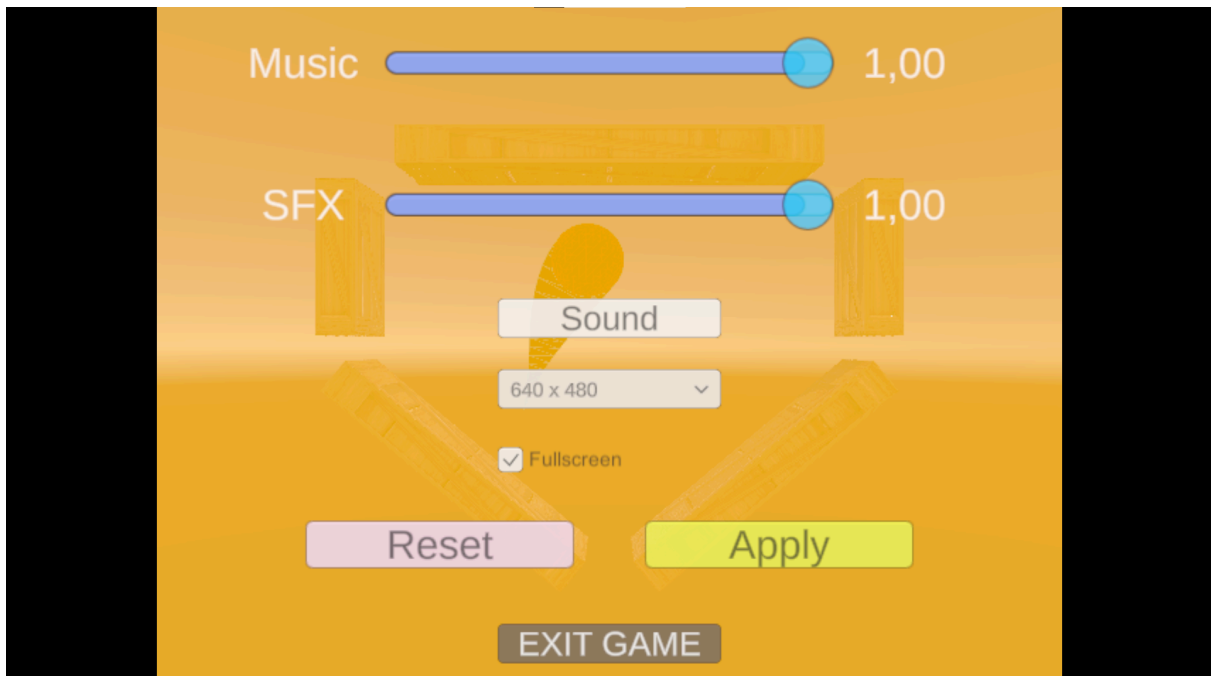
No se rompe en ninguna de las resoluciones (monitor instituto)

Se han comprobado todas las resoluciones disponibles en el monitor del instituto, desde las resoluciones predeterminadas del sistema, hasta las que suelen dar más problemas. Aquí se muestran fotos de los dos resultados en cuestión.

resolución 1920 x 1080 (predeterminado/recomendado del sistema)



resolución 640 x 480 (de los que más problemas suele dar)



EXTRAS

Default Values

Utilizamos un método largo llamado DefaultValues(), su función es recoger todos esos valores temporales que han sido modificados o no, y volverles a aplicar una configuración predeterminada, pudiendo así también bajo la misma lógica actualizar toda la interfaz.

```
1 referencia
private void DefaultValues()
{
    //valores predeterminados (default)
    tempMusicVolume = 1.0f;
    tempSFXVolume = 1.0f;
    tempFullscreen = true;

    //actualizar sliders con los valores predeterminados
    musicSlider.value = tempMusicVolume;
    sfxSlider.value = tempSFXVolume;

    //actualizar audio
    musicAudioSource.volume = tempMusicVolume;
    sfxAudioSource.volume = tempSFXVolume;
    ballAudio.volume = tempSFXVolume;

    //actualizar textos
    musicVolumeText.text = tempMusicVolume.ToString("0.00");
    sfxVolumeText.text = tempSFXVolume.ToString("0.00");

    //actualizar fullscreen
    toggleScreen.isOn = tempFullscreen;
    Screen.fullScreenMode = FullScreenMode.FullScreenWindow;

    //obtenemos la resolución predeterminada del dispositivo
    Resolution defaultResolution = Screen.currentResolution;
    //actualizamos el dropdown con la resolución predeterminada
    tempResolutionWidth = defaultResolution.width;
    tempResolutionHeight = defaultResolution.height;
    //ahora si, aplicamos la resolución predeterminada
    Screen.SetResolution(tempResolutionWidth, tempResolutionHeight, Screen.fullScreen);

    for (int i = 0; i < resolutionDropdown.options.Count; i++)
    {
        if (resolutionDropdown.options[i].text ==
            tempResolutionWidth + " x " + tempResolutionHeight)
        {
            resolutionDropdown.value = i; //aplicar la resolución
            break; //salir del bucle al encontrarla
        }
    }
    resolutionDropdown.RefreshShownValue(); //actualizar el valor mostrado en el dropdown

    Debug.Log("Default values applied");
}
```

Botón Apply

Todos los valores con los que hemos trabajado y que tenemos que guardar, ahora en vez de guardarlos directamente en PlayerPrefs al hacer cambios, los podemos retener de forma temporal y asegurarnos de aplicarlos solo si así lo deseamos. Utilizando las variables temp para almacenar y finalmente descargar esos valores, pasándolos a PlayerPrefs

```
//variables temporales que representan los valores que el usuario está modificando en el menú
private float tempMusicVolume;
private float tempSFXVolume;
private int tempResolutionWidth;
private int tempResolutionHeight;
private bool tempFullscreen;
```

1 referencia

```
private void ApplySettings()
{
    //al aplicar los cambios, guardamos los valores temporales en PlayerPrefs
    PlayerPrefs.SetFloat("MusicVolume", tempMusicVolume);
    PlayerPrefs.SetFloat("SFXVolume", tempSFXVolume);

    //PlayerPrefs.SetInt("ResolutionIndex", index);
    //en vez de guardar el index, es mejor guardar la resolución en sí
    //por si cambian las resoluciones disponibles entre dispositivos
    PlayerPrefs.SetInt("ResolutionWidth", tempResolutionWidth);
    PlayerPrefs.SetInt("ResolutionHeight", tempResolutionHeight);

    //guardamos el estado del toggle en PlayerPrefs
    //tenemos que utilizar un int porque PlayerPrefs no soporta bools
    PlayerPrefs.SetInt("IsFullscreen", tempFullscreen ? 1 : 0); //1 = true, 0 = false

    PlayerPrefs.Save(); //nos aseguramos de guardar los cambios (por si acaso)

    Debug.Log("Settings applied correctly");
}
```

Ventana de advertencia

Para la Ventana de advertencia con un warning que nos avisa si tenemos cambios sin guardar tenemos diversos métodos diferentes que trabajan entre sí. El método HasUnsavedChanges() se llama en una condición durante OnPauseMenu(), donde cuando se cierra el menú, primero se comprueba si tenemos cambios sin guardar utilizando este mismo método, y si es así se llama al método ShowWarning() el cual muestra esta ventana de advertencia. Esta ventana se trata de otro objeto con otro canvas group, es útil porque desde código podemos bloquear los raycasts cuando no lo necesitamos. Dentro contiene dos botones, uno para salir al menú sin guardar nada y el otro para cerrar la ventana de warning pero seguir en el menú de pausa.

```
else //si el menu de pausa esta abierto, lo cerramos
{
    //pero, antes de cerrarlo, comprobamos si hay cambios sin guardar
    if (HasUnsavedChanges())
    {
        ShowWarning();
    }
    else
    {

```

```

1 referencia
private bool HasUnsavedChanges()
{
    float savedMusicVolume = PlayerPrefs.GetFloat("MusicVolume", 1.0f);
    float savedSFXVolume = PlayerPrefs.GetFloat("SFXVolume", 1.0f);

    int savedWidth = PlayerPrefs.GetInt("ResolutionWidth", Screen.currentResolution.width);
    int savedHeight = PlayerPrefs.GetInt("ResolutionHeight", Screen.currentResolution.height);

    int savedFullscreen = PlayerPrefs.GetInt("IsFullscreen", 1);
    bool savedFullscreenValue = savedFullscreen == 1;

    //aquí comprobamos si los valores temporales son diferentes a los guardados en PlayerPrefs
    if (tempMusicVolume != savedMusicVolume)
        return true; //si el valor temporal es diferente al guardado, hay cambios sin guardar

    if (tempSFXVolume != savedSFXVolume)
        return true;

    if (tempResolutionWidth != savedWidth ||
        tempResolutionHeight != savedHeight)
        return true;

    if (tempFullscreen != savedFullscreenValue)
        return true;

    return false; //si no hay diferencias, no hay cambios sin guardar
}

```

```

1 referencia
private void ShowWarning() //mostramos y permitimos interactuar con el canvas de warning
{
    //evitamos poder interactuar con el canvas de pausa mientras el warning esta activo
    canvasFader.GetCanvasGroup().interactable = false;

    warningCanvasGroup.alpha = 1.0f;
    warningCanvasGroup.interactable = true;
    warningCanvasGroup.blocksRaycasts = true;
}

```

```

2 referencias
private void HideWarning() //volvemos a ocultar y desactivar el canvas de warning
{
    warningCanvasGroup.alpha = 0.0f;
    warningCanvasGroup.interactable = false;
    warningCanvasGroup.blocksRaycasts = false;

    //volvemos a permitir interactuar con el canvas de pausa
    canvasFader.GetCanvasGroup().interactable = true;
}

```

```

//si cancelamos, entonces se cierra el warning y nos quedamos aun en el menu de pausa
1 referencia
private void CancelExit()
{
    HideWarning();
}

//si confirmamos, entonces se cierra el warning y salimos del menu de pausa
1 referencia
private void ConfirmExit()
{
    HideWarning();

    canvasFader.FadeOut(0.5f);
    Time.timeScale = 1.0f;
    canvasFader.GetCanvasGroup().interactable = false;
    isOpen = false;
}

```

